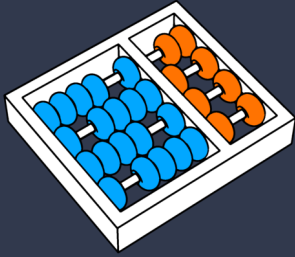




UNICAMP



Ferramentas e Integrações com Agentes

Prof. Dr. Julio Cesar dos Reis
dosreis@unicamp.br

Agenda

- ❑ Fundamentos das Tools em Agentic AI
- ❑ Ferramentas como extensão do agente
- ❑ Ciclo Ação-Percepção
- ❑ Padrões de integração
- ❑ Segurança e Governança

Motivação

- ❏ *"Descubra a cotação atual do dólar e envie um e-mail para minha equipe com essa informação"*
- ❏ Responder corretamente exige duas capacidades distintas:
 - consultar uma informação atualizada na Internet;
 - enviar um e-mail por meio de um serviço externo.

Outras tarefas

- ❏ Considere ainda as seguintes tarefas:
 - consultar um banco de dados corporativo;
 - executar um script Python;
 - reservar uma passagem aérea;
 - gerar um gráfico a partir de dados;
 - realizar uma revisão bibliográfica.
- ❏ Exigem acesso a recursos que extrapolam o modelo de linguagem

Problema

- ❑ Nenhuma dessas capacidades faz parte do conhecimento interno do modelo
- ❑ O LLM pode compreender a solicitação, planejar a sequência de ações e produzir texto
- ❑ Mas não possui acesso direto a serviços externos

Fundamentos

Ferramenta

- ❑ **Tool** representa uma capacidade reutilizável do sistema
- ❑ Sua implementação pode variar desde:
 - uma simples função
 - workflows complexos
 - agentes especializados ou
 - serviços remotos
- ❑ Todos acessados por meio de uma interface uniforme
- ❑ Implementação é transparente ao agente

Ferramenta

- ❑ Tool é uma abstração arquitetural
- ❑ Encapsulamento de capacidades (o sistema sabe fazer algo)
- ❑ Workflow é uma possível implementação de uma Tool
- ❑ Agentes podem utilizar outras Tools
- ❑ Uma Tool pode utilizar outras Tools
- ❑ LLM utiliza ferramentas para ampliar suas capacidades

Problema que resolve

- ❏ Sem ferramentas, o agente é:
 - limitado ao conhecimento aprendido durante o treinamento e à geração de texto
- ❏ Com ferramentas, o agente passa a:
 - acessar informações atualizadas;
 - executar ações;
 - integrar sistemas;
 - utilizar serviços especializados;
 - delegar tarefas complexas.

Problema que resolve

- ❑ Separa responsabilidade funcional da implementação técnica
- ❑ Evita que agentes dependam diretamente da implementação interna das funcionalidades.
- ❑ Isso facilita:
 - manutenção;
 - reutilização;
 - evolução independente;
 - testes;
 - escalabilidade.

Interface × Implementação

- ❑ **Interface**: É aquilo que o agente conhece.
 - entradas; saídas; contrato; tratamento de erros.
- ❑ **Implementação**: É o mecanismo interno responsável pela execução.
 - múltiplos nós; workflows; chamadas de API; bancos de dados;
 - loops; agentes especializados.
 - O agente não deve depender desses detalhes.

Ferramentas simples vs. Tools “Inteligentes”

- **Interface uniforme**

Garante um padrão consistente de comunicação para todas as ferramentas.

- **Complexidade encapsulada**

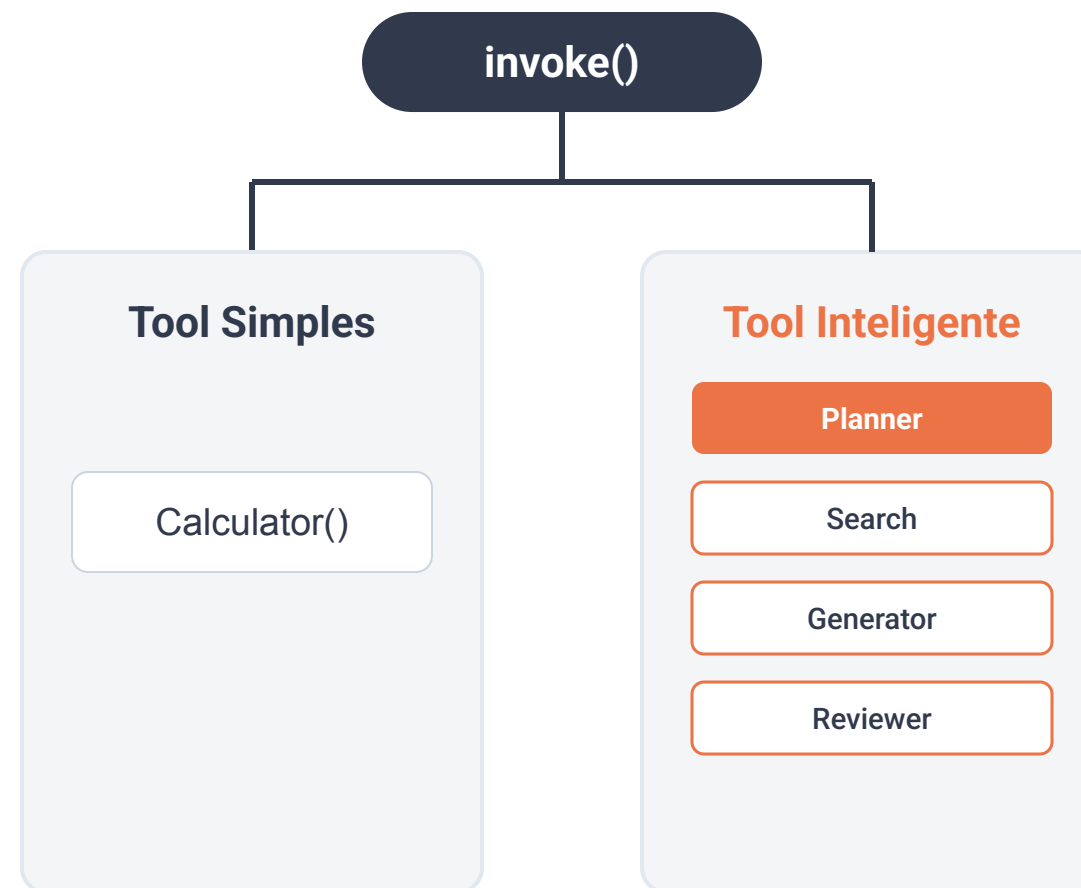
Oculta fluxos complexos, APIs e regras de negócio do agente principal.

- **Reutilização**

Permite que diferentes agentes consumam a mesma funcionalidade de forma idêntica.

- **Evolução transparente**

Mudanças internas na ferramenta não quebram a integração com o agente.



Categorias de Tools

- ❏ Reasoning Tool
- ❏ Planning Tool
- ❏ Retrieval Tool
- ❏ Action Tool
- ❏ Communication Tool
- ❏ Integration Tool
- ❏ Monitoring Tool

Projetando uma Tool

Projetando uma Tool

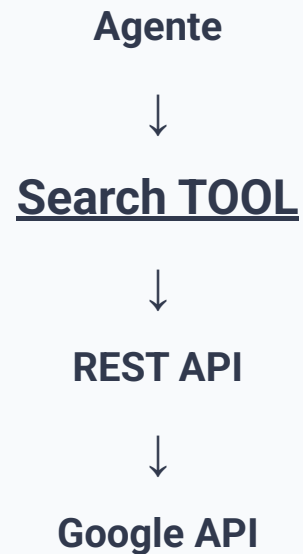
- ❑ Resolve uma única responsabilidade?
- ❑ Possui uma interface simples?
- ❑ Pode ser reutilizada em outros contextos?
- ❑ Está desacoplada do agente?
- ❑ Sua implementação pode evoluir sem alterar quem a utiliza?

Uma boa Tool

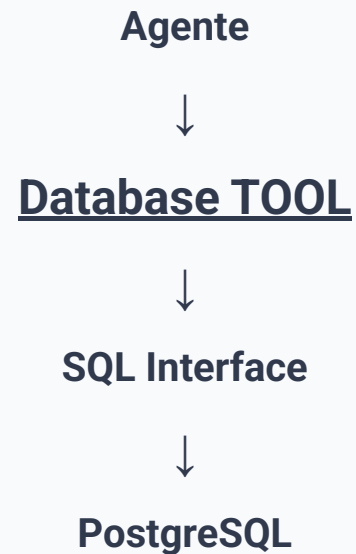
- ❑ **alta coesão:** resolve um único problema de forma clara;
- ❑ **baixo acoplamento:** depende o mínimo possível de outros componentes;
- ❑ interface simples e estável;
- ❑ encapsulamento da lógica interna;
- ❑ reutilização em diferentes agentes e workflows;
- ❑ facilidade de teste e observabilidade;
- ❑ evolução independente.

Tool é para integração

Search Tool Flow



Database Tool Flow



Research Tool (*Composite*)

-  Search Tool
-  Citation Tool
-  Summarizer Tool

Cada Tool resolve apenas um problema.
Não precisa executar tudo sozinha.
Pode delegar responsabilidades.

Anatomia de uma Ferramenta

Identificação

Nome

Identificador semântico e exclusivo da ferramenta para invocação do agente.

Descrição

Explicação detalhada do propósito e das capacidades para o modelo decidir quando usá-la.

Interface

Entrada & Saída

Esquemas de dados rígidos (schemas) que definem os inputs aceitos e o formato de retorno.

Contrato

Garantias operacionais, regras de negócios e limites de utilização.

Documentação

Exemplos práticos e guias para orientar o comportamento de uso do LLM.

Operação

Implementação

Código lógico e integrações reais que executam a ação solicitada.

Tratamento de Erros

Tratamento de falhas e respostas inteligíveis para autocorreção do agente.

Observabilidade & Métricas

Monitoramento, auditoria, latência e dados analíticos de consumo da ferramenta.

possui comportamento semelhante ao de uma API Rest

Tool Registry

- ❏ Em aplicações reais existem dezenas de Tools
- ❏ O agente precisa conhecer quais estão disponíveis
- ❏ Cada Tool possui metadados.
 - nome (claro*)
 - descrição (objetiva*)
 - entradas (consistentes*)
 - saídas (consistentes*)
 - exemplos (úteis*)
- ❏ *Boas práticas

Tool Calling

② O Cenário

Exemplo de Pergunta:

"Faça uma revisão bibliográfica sobre Agentic AI."

O Desafio:

Como o agente sabe quais ferramentas (Tools) deve utilizar para responder?

Tool Calling

② O Cenário

Exemplo de Pergunta:

"Faça uma revisão bibliográfica sobre Agentic AI."

O Desafio:

Como o agente sabe quais ferramentas (Tools) deve utilizar para responder?

∩ Tomada de Decisão

1. Pergunta (Entrada)
- ↓
2. LLM (Decisão cognitiva)
- ↓
3. Qual Tool? (Seleção semântica)
- ↓
4. Invoke() (Chamada)
- ↓
5. Resultado (Retorno de dados)

Tool Calling

② O Cenário

Exemplo de Pergunta:

"Faça uma revisão bibliográfica sobre Agentic AI."

O Desafio:

Como o agente sabe quais ferramentas (Tools) deve utilizar para responder?

🔗 Tomada de Decisão

1. Pergunta (Entrada)
- ↓
2. LLM (Decisão cognitiva)
- ↓
3. Qual Tool? (Seleção semântica)
- ↓
4. Invoke() (Chamada)
- ↓
5. Resultado (Retorno de dados)

🔄 Ciclo de Execução

1. Usuário
- ↓
2. LLM
- ↓
3. Planejamento
- ↓
4. Tool Calling
- ↓
5. Execução
- ↓
6. Observação
- ↓
7. Resposta

Tool Discovery

 Estática

Agent



Lista fixa de Tools

 Dinâmica

Agent



Registry



Discovery



Invoke

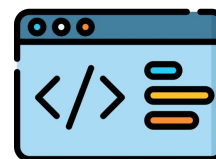
Workflow × Tool × Agent

Workflow	Tool	Agent
Sequência de execução	Capacidade reutilizável	Entidade que decide
Não decide	Não decide	Decide
Executa passos	Expõe uma interface	Planeja
Pode virar uma Tool	Pode encapsular Workflows	Usa Tools

Ferramentas como Extensão do Agente

Ferramentas como Extensão do Agente

- ❑ Um agente deixa de ser apenas um gerador de texto quando passa a acionar sistemas externos para obter dados
- ❑ Executar operações (ou produzir efeitos no ambiente)
 - funciona como adaptadores entre agentes e recursos externos
- ❑ Busca em base de dados
- ❑ Navegação Web
- ❑ Escrita de documentos
- ❑ Chamadas de funções
- ❑ Escrita de código



Da resposta textual à ação operacional

- ❑ A integração com ferramentas transforma decisões do agente em chamadas controladas
- ❑ O resultado dessas chamadas retorna como observação e passa a fazer parte (atualizar) do estado.

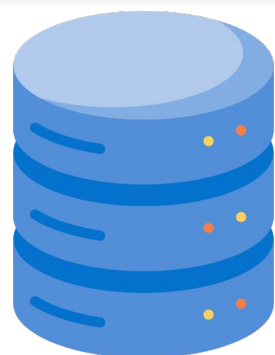


Tipos de ferramentas externas

- ❑ Ferramentas são interfaces entre o grafo do agente e recursos externos
- ❑ Cada tipo amplia uma capacidade operacional diferente.



Serviços web, buscas,
sistemas internos, clima,
tickets



Dados

Bancos SQL/NoSQL,
vetores, documentos,
logs



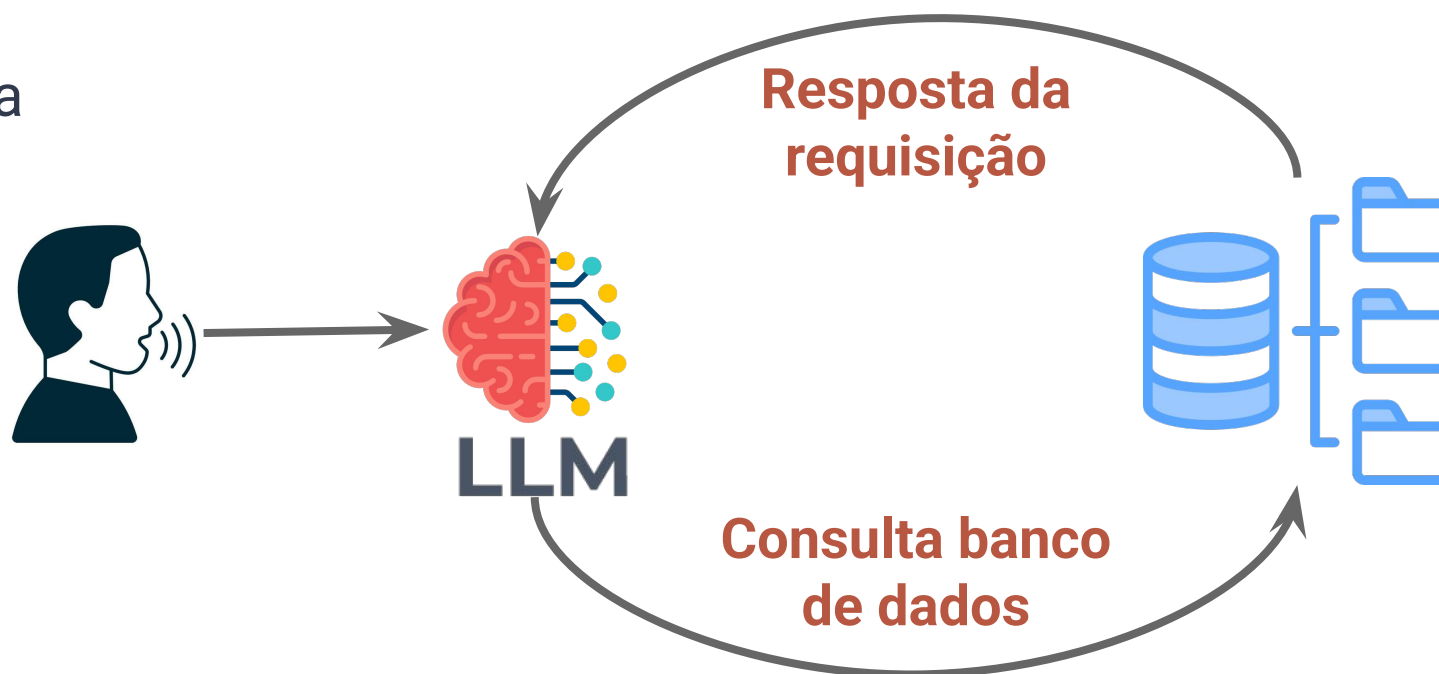
Computação

Cálculos, execução de
código, scripts, jobs
assíncronos

Ferramentas de Dados

❏ O agente normalmente não deve “inventar” conhecimento

- Consulta uma fonte
- Interpretar o retorno
- Produzir uma resposta



Ferramentas que produzem efeitos

- ❑ Ferramentas podem efetuar modificações em sistemas
 - Não apenas retornam informação
- ❑ Nesses casos, o desenho do agente precisa ser mais conservador

Exemplos

- ❑ Criar uma tarefa em um sistema
- ❑ Enviar uma mensagem
- ❑ Atualizar uma base
- ❑ Executar um job de processamento

Cuidados

- ❑ Confirmar intenção do usuário
- ❑ Validar parâmetros

Composição de Tools

- ❏ Uma Tool não precisa executar tudo sozinha.
 - Ela pode delegar responsabilidades

❏ **Research Tool**

❏ Internamente

- *Search Tool*
- *Citation Tool*
- *Summarizer Tool*

- ❏ Cada Tool resolve apenas um problema

Benefícios

- ❏ Maior reutilização
- ❏ Menor duplicação
- ❏ Especialização
- ❏ Testabilidade

Exemplo

- ❑ LLMs produzem bons resultados em geração de texto
- ❑ Porém, por exemplo, sua saída probabilística prejudica ao tratar problemas matemáticos.

= ChatGPT 4o



What is bigger 9.11 or 9.9

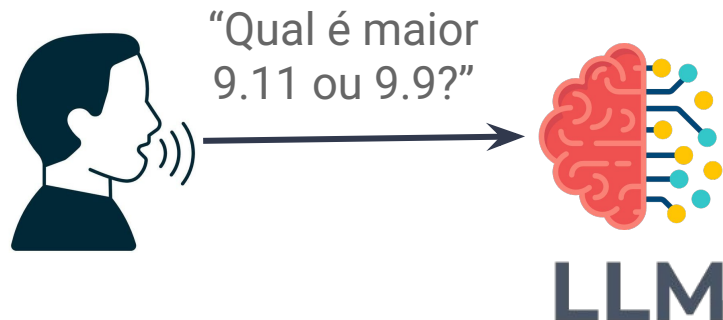


9.9 is bigger than 9.11.

When comparing these numbers as decimals, 9.11 is equivalent to 9.110, which is less than 9.9 (equivalent to 9.900). Hence, 9.9 is the larger number.

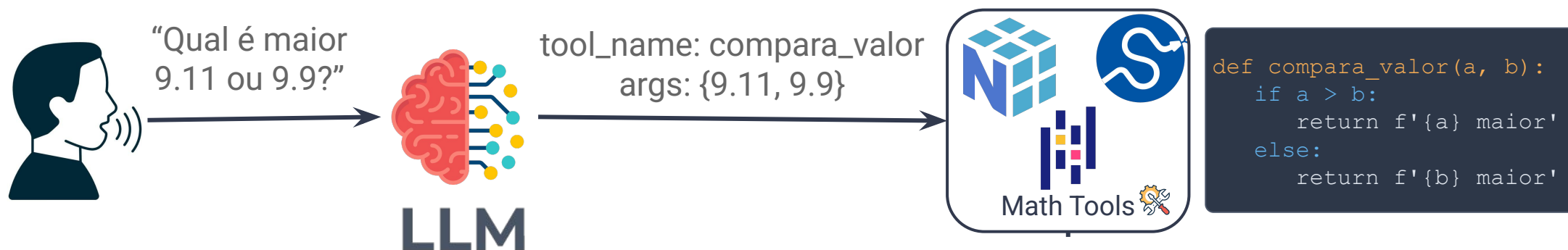
Exemplo

- ❏ Podemos chamar ferramentas para resolver esses problemas



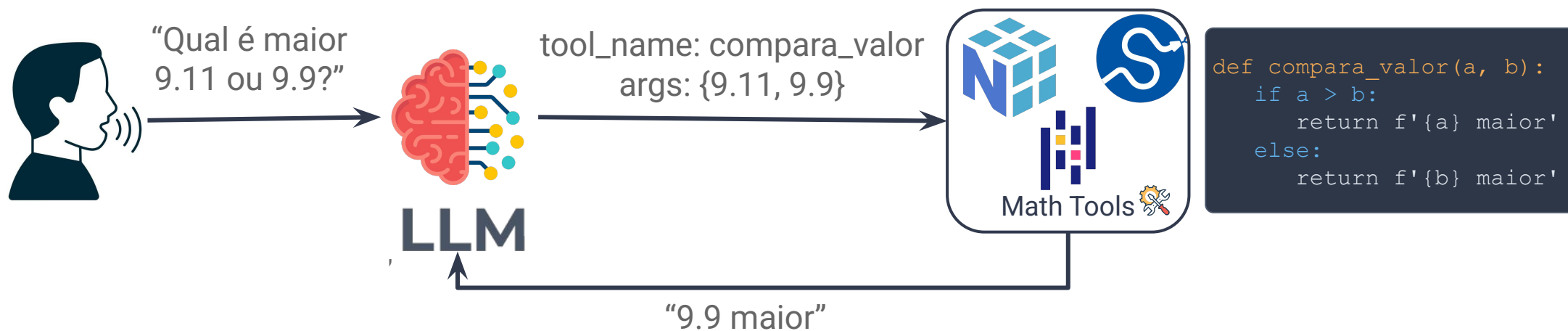
Exemplo

- ❑ LLM decide qual tool chamar para melhor resolver a questão de entrada (em vez de responder diretamente)
- ❑ A tool executa com os argumentos adequados



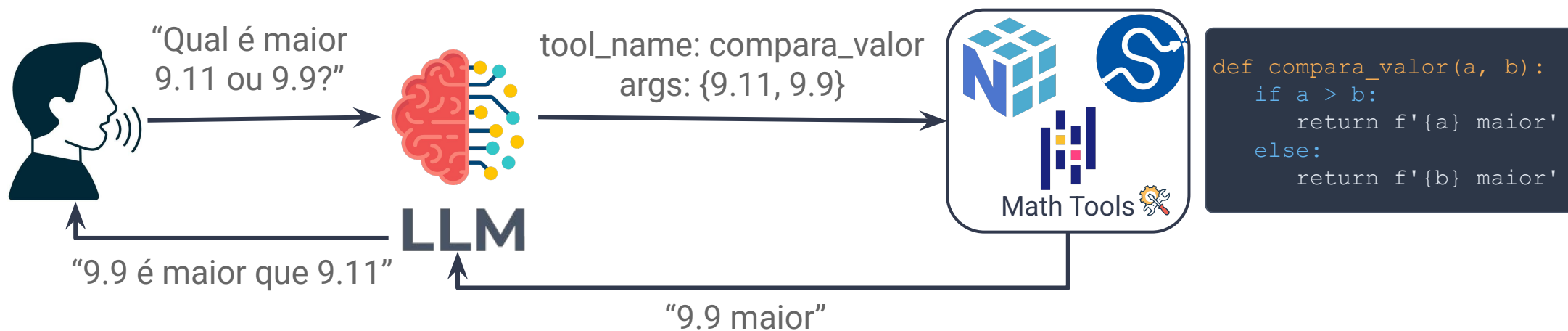
Exemplo

- ❑ LLM decide qual tool chamar para melhor resolver a questão de entrada (em vez de responder diretamente)
- ❑ A tool executa com os argumentos adequados
- ❑ Gera o resultado



Exemplo

- ❑ LLM decide qual tool chamar para melhor resolver a questão de entrada (em vez de responder diretamente)
- ❑ A tool executa com os argumentos adequados
- ❑ Gera o resultado
- ❑ Retorna ao usuário



Ciclo Ação-Percepção

Ciclo Ação-Percepção

- ❏ O uso de ferramenta normalmente ocorre em ciclo
- ❏ O agente decide uma ação e executa
- ❏ Observa o retorno
- ❏ Atualiza seu raciocínio



se ainda faltar informação ou houver erro

Interpretação dos Resultados

- ❑ O retorno de uma ferramenta não deve ser tratado automaticamente como resposta final
- ❑ Precisa ser interpretado, validado e contextualizado

O agente deve verificar

- ❑ O resultado responde à pergunta?
- ❑ Há erro, vazio ou inconsistência?
- ❑ O formato está correto?
- ❑ A resposta precisa de explicação?

Quando o agente deve usar uma ferramenta?

- ❑ A decisão de chamar uma ferramenta pode ser modelada como roteamento condicional
- ❑ O estado indica:
 - se a resposta exige informação externa ou
 - ação operacional

Quando o agente deve usar uma ferramenta?

Sinais para usar/chamar:

- ❏ Necessidade de dado atualizado
- ❏ Pedido de cálculo exato
- ❏ Consulta a sistema privado
- ❏ Ação com efeito externo

Sinais para não usar/chamar:

- ❏ Pergunta conceitual simples
- ❏ Informação já presente no estado
- ❏ Ferramenta sem permissão
- ❏ Parâmetros incompletos

Padrões de Integração

Padrões de Integração

- ❑ Ferramenta não deve ser vista apenas como uma função Python solta
- ❑ Precisa de contrato, adaptação e controle de erro
- ❑ Para o LLM usar uma ferramenta corretamente, a ferramenta precisa ser descrita como um contrato explícito
 - Nome, finalidade, entradas esperadas e formato de saída.

Padrões de Integração

- ❑ A escolha do padrão depende do grau de autonomia desejado
- ❑ Do risco das ferramentas e da necessidade de controle sobre o fluxo

Tool Router

Tool Node

Supervisor

Human-in-the-loop

Tool Router

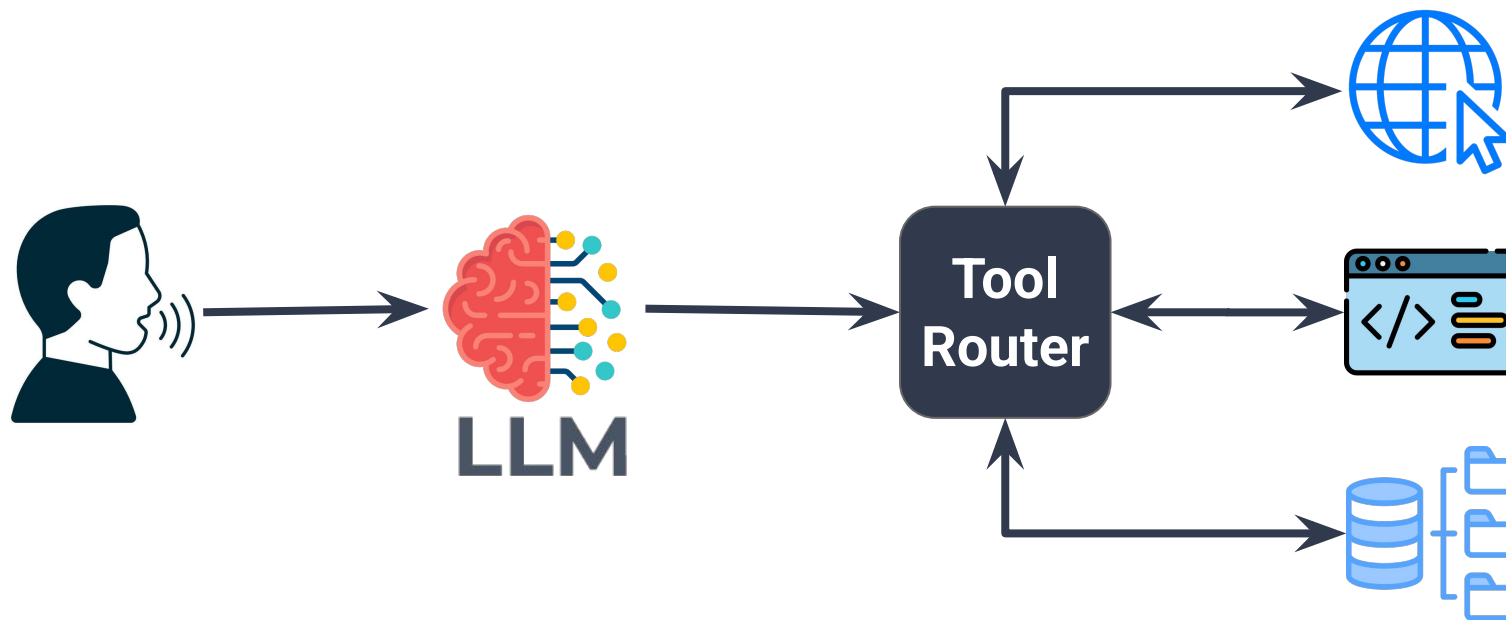
- ❑ Escolhe qual das ferramentas será utilizada em cada momento do *reasoning*.

Tool Router

Escolhe a ferramenta ideal baseando-se no estado atual do sistema

Quando usar

Múltiplas fontes de dados ou serviços integrados.



Tool Node

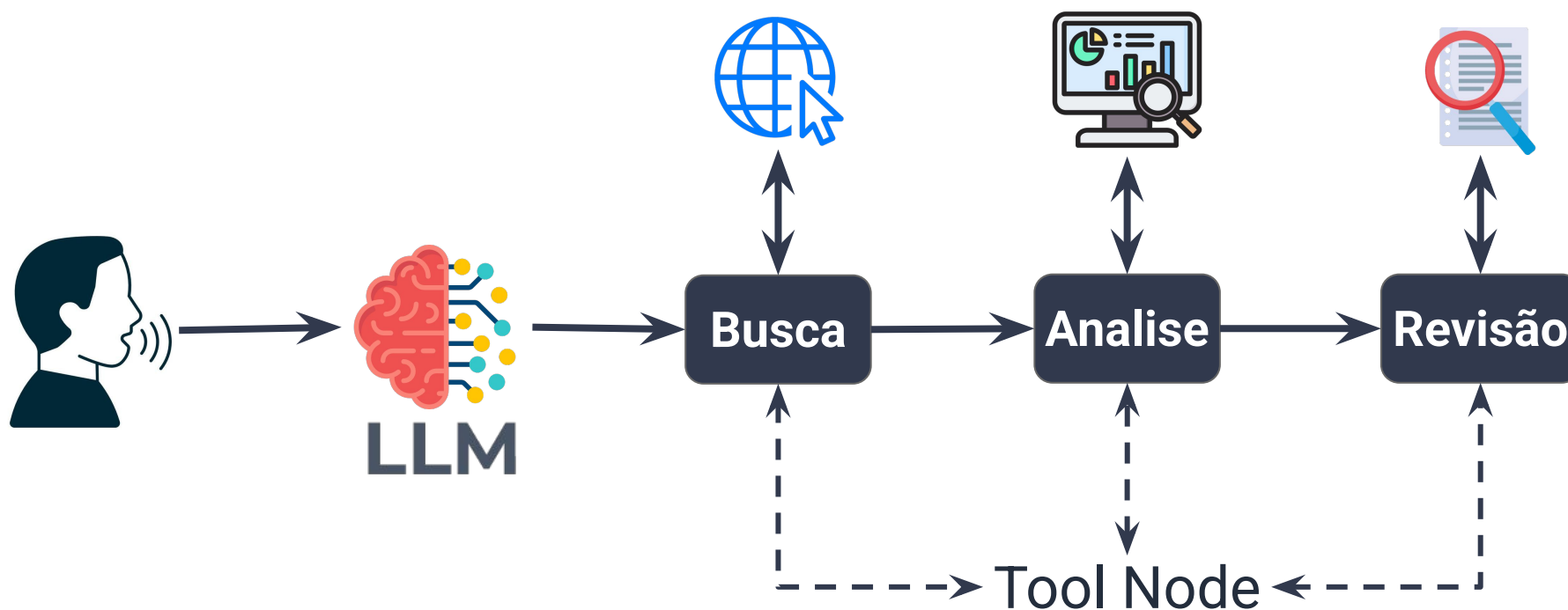
- ❑ Nós responsáveis por chamar uma ferramenta específica

Tool Node

Nó dedicado que executa uma ferramenta específica de forma isolada.

Quando usar

Necessidade de integração previsível e altamente testável.



Supervisor

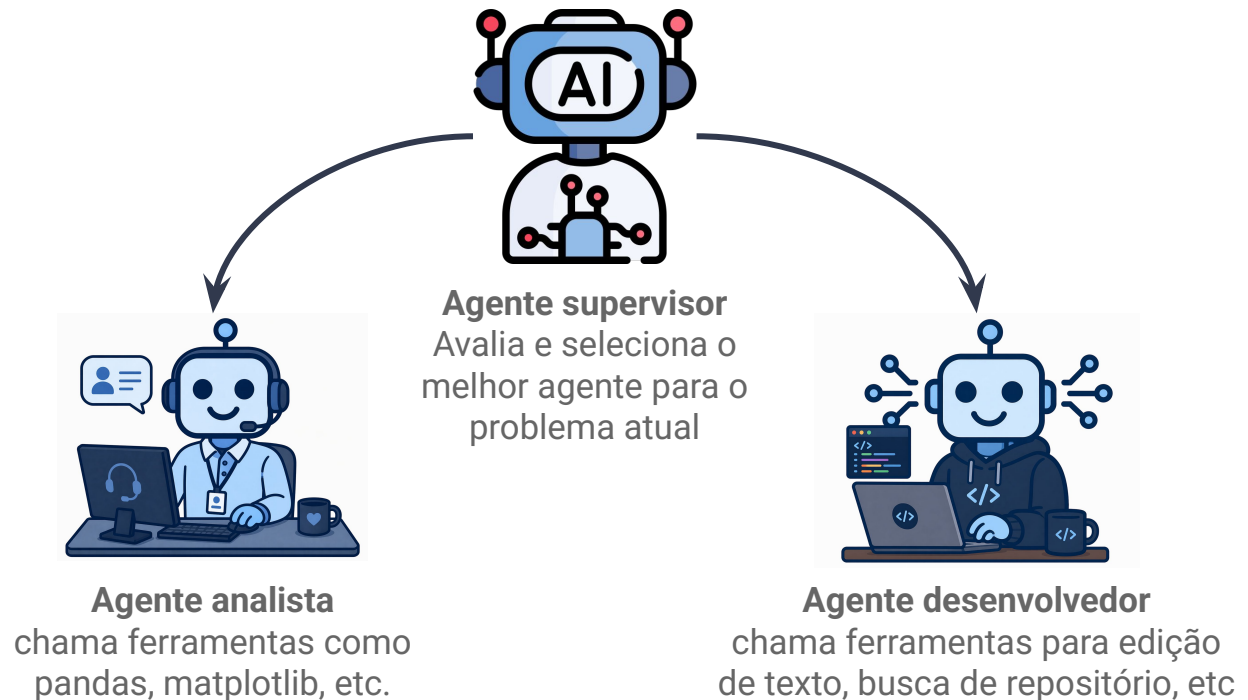
- ❑ Orquestra quais ferramentas serão utilizadas através de agentes especialistas

Supervisor

Coordena múltiplos agentes especialistas e suas respectivas ferramentas.

Quando usar

Tarefas compostas e fluxos de trabalho complexos.



Human-in-the-loop

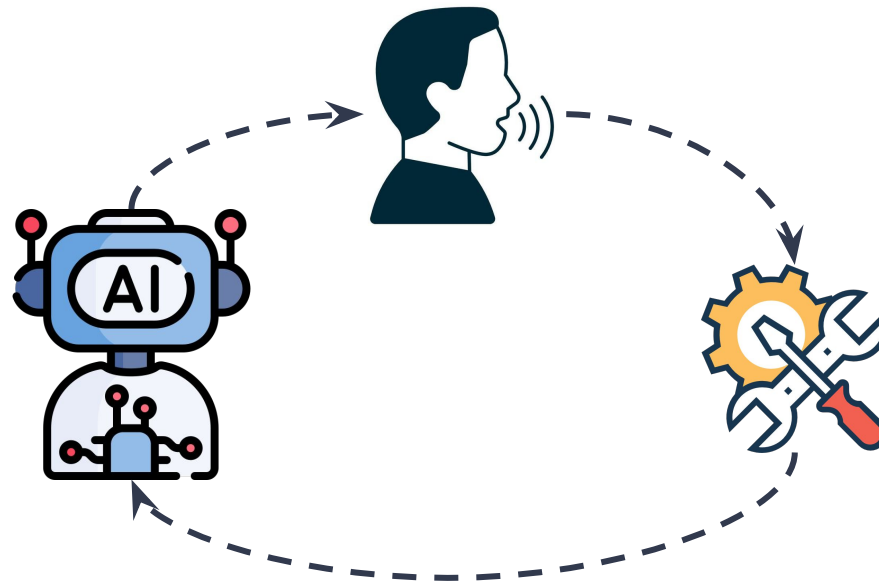
- ❑ Ação humana direta para realização de uma tarefa sensível ou que requeira autorização para acessar partes do sistema.

Human-in-the-loop

Interrupção estratégica para validação humana antes de ações críticas.

Quando usar

Operações de alto risco ou impacto irreversível.



Segurança & Governança

Validação, permissões e limites

- ❑ Ferramentas ampliam a capacidade do agente
- ❑ Podem também ampliar o risco
- ❑ A integração deve impor limites antes, durante e depois da chamada.

Antes

Validar intenção,
permissões e parâmetros

Durante

Timeout e limites de custo

Depois

Registrar resultado e
tratar erro

Tratamento de Erro e Observabilidade

- ❑ Em workflows reais, ferramentas falham.
- ❑ O agente precisa transformar falhas técnicas em decisões controladas

Estratégias

- ❑ Fallback para outra fonte
- ❑ Mensagem clara ao usuário
- ❑ Pausa para intervenção humana

Registrar

- ❑ Qual ferramenta foi chamada
- ❑ Quais argumentos foram usados
- ❑ Qual retorno foi obtido
- ❑ Quanto tempo/custo foi gasto

Síntese

- ❑ LLMs possuem capacidades com foco em raciocínio e geração de texto
- ❑ Muitas tarefas exigem interação com sistemas externos
- ❑ Ferramentas ampliam as capacidades do agente
- ❑ O agente decide e a Tool executa
- ❑ Interface separada da implementação (pode variar)
- ❑ Agente não conhece detalhes internos de implementação
- ❑ Resultados de ferramentas entram como observações que devem ser interpretadas antes da resposta final